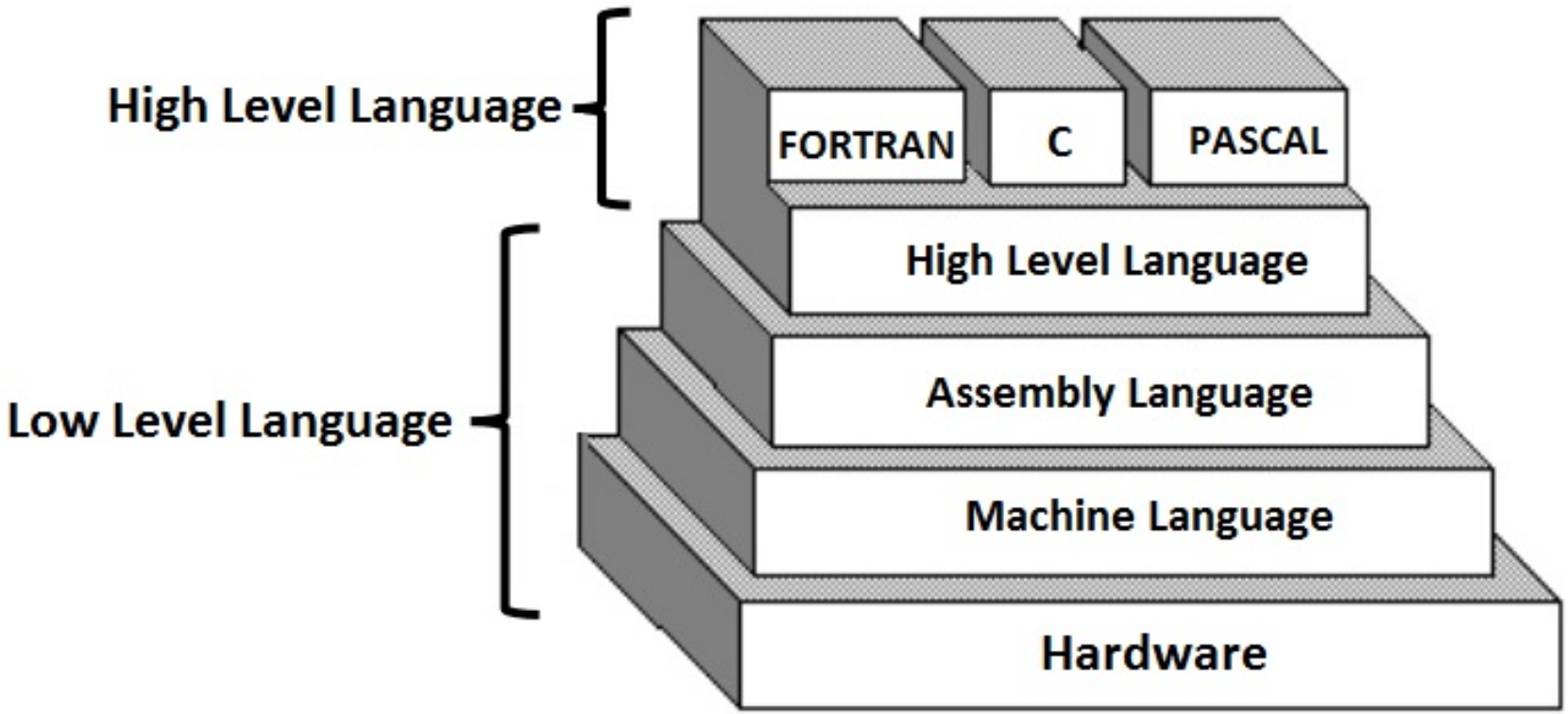
Introduction of computer languages

- A language is the main medium of communicating between the Computer systems and the most common are the programming languages.
- As we know a **Computer only understands binary numbers that is 0 and 1 to perform various operations** but the languages are developed for different types of work on a Computer.
- A language consists of all the instructions to make a request to the system for processing a task.
- From the **first generation and now fourth generation of the Computers there were several programming languages used to communicate with the Computer**..

Computer Language Description:

- A Computer language includes various languages that are used to communicate with a Computer machine.
- Some of the languages like programming language which is a **set of codes or instructions used for communicating the machine.**
- Machine code is also considered as a computer language that can be used for programming.
- And also **HTML which is a computer language or a markup language but not a programming language.**
- Similarly there are different types of languages developed for different types of work to be performed by communicating with the machine.
- But all the languages that are now available are categorized into **two basic types of languages including Low-level language and High level language**.



Computer Language and its Types

High-Level Language	Low-Level Language
It is a programming language in the sense that it's user-friendly.	It is a type of language, which is machine-friendly.
These languages are considered less memory efficient.	These languages are considered high memory efficient.
It is easy to understand.	It's difficult to understand.
It's straightforward to troubleshoot.	Comparatively, it is complex to debug.
It's easy to keep up with.	In comparison, it is difficult to maintain.
High-level language is portable.	Low-level language is non-portable.
It is compatible with all operating systems.	It is entirely dependent on the machine.
For translation, a compiler or interpreter is required.	For translation, it will need an assembler.
It is commonly used in programming.	It is no longer widely used in programming.

- **Machine level Language :** Machine language is lowest level of programming language. It handles binary data i.e. **0's** and **1's.** It directly interacts with system. Machine language is difficult for human beings to understand as it comprises combination of 0's and 1's. There is software which translate programs into machine level language.
- **Assembly Level Language :** Assembly language is a middle-level language. It consists of a set of instructions in a specific format called **commands.** It uses symbols to represent field of instructions. It is very close to machine level language. The computer should have assembler to translate assembly level program to machine level program. Examples include ADA, PASCAL, etc. It is in human-readable format and takes lesser time to write a program and debug it. However, it is a machine dependent language.
- **High-level Language :** High-level language uses format or language that is most familiar to users. The instructions in this language are called **codes** or **scripts**. The computer needs a compiler and interpreter to convert high-level language program to machine level language. Examples include C++, Python, Java, etc. It is easy to write a program using high level language and is less time-consuming.

What is the programming concept?

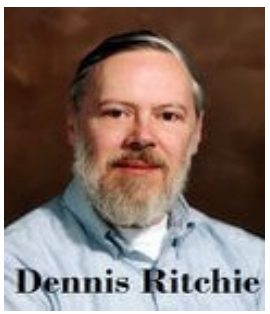
- In the context of computing, programming means **creating a set of instructions not for a person but for a computer, in order to accomplish a specific task**. To do so you use a set of directives—a programming language—known to both the programmer and the computer operating system.
- Programming is **writing computer code to create a program, to solve a problem**. Programs are created to implement algorithms . Algorithms can be represented as pseudocode or a flowchart , and programming is the translation of these into a computer program.

Here are the 4 basic concepts of any programming language:

- Variables.
- Control Structures.
- Data Structures.
- Syntax.

History of C Language

- **C programming language** was developed in 1972 by Dennis Ritchie at bell laboratories of AT&T (American Telephone & Telegraph), located in the U.S.A.
- **Dennis Ritchie** is known as the founder of the c language.



- It was developed to overcome the problems of previous languages such as B, BCPL, etc.
- The UNIX operating system development started in the year 1969, and its code was rewritten in C in the year 1972.

key advantages of learning C Programming:

- Easy to learn
- Structured language
- It produces efficient programs
- It can handle low-level activities
- It can be compiled on a variety of computer platforms

Difference between traditional and modern C

Traditional (ANSI C)	Modern c (C Language)
American National Standards Institute formulated ANSI C.	Dennis Ritchie developed C programming language.
ANSI C is only for C programming, for example, Syntax and Semantics are programs, written in C language with the help of ANSI C.	C language is one of the most used programming languages. But ANSI C is the set of standards. Many applications make use of the C programming language.
ANSI C is just a version of the C programming language.	C is a programming language
However, to add some functions in the C language, ANSI C evolved.	In order to develop different types of programs, we utilize C language.
The type signed char, type void, signed type qualifier, unsigned qualifier, a unary positive sign, type long double, const qualifier and enumeration type are available in ANSI C	The type signed char, type void, signed type qualifier, unsigned qualifier, a unary positive sign, type long double, const qualifier and enumeration type are not available in C.

Character Set in C

- In the C programming language, the character set refers to a set of all the valid characters that we can use in the source program for forming words, expressions, and numbers.
- Here is a table that represents all the types of character sets that we can use in the C language.

1. Lowercase Alphabets
2. Uppercase Alphabets
3. Digits
4. Special Characters
5. White Spaces

Type of Character	Description	Characters
Lowercase Alphabets	a to z	a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z
Uppercase Alphabets	A to Z	A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z
Digits	0 to 9	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Special Characters	–	` ~ @ ! $ # ^ * % & ( ) [ ] { } < > + = _ – | / \ ; : ‘ “ , . ?
White Spaces	–	Blank Spaces, Carriage Return, Tab, New Line

printf() and scanf() in C

- The printf() and scanf() functions are used for input and output in C language. Both functions are inbuilt library functions, defined in stdio. h (header file).

1. printf() function

The **printf() function** is used for output. It prints the given statement to the console.

- The syntax of printf() function is given below:

  printf("format string",argument_list);

- The **format string** can be %d (integer), %c (character), %s (string), %f (float) etc.

2. scanf() function

- The **scanf() function** is used for input. It reads the input data from the console.

  scanf("format string",argument_list);

First C program

```
#include <stdio.h>

int main() {
  printf("Hello World!");
  return 0;
}
```

Example explained

Line 1: #include <stdio.h> is a **header file library** that lets us work with input and output functions, such as printf() (used in line 4).
Header files add functionality to C programs.

Line 2: A blank line. C ignores white space. But we use it to make the code more readable.

Line 3: Another thing that always appear in a C program, is main(). This is called a **function**. Any code inside its curly brackets {} will be executed.

Line 4: printf() is a **function** used to output/print text to the screen. In our example it will output "Hello World".

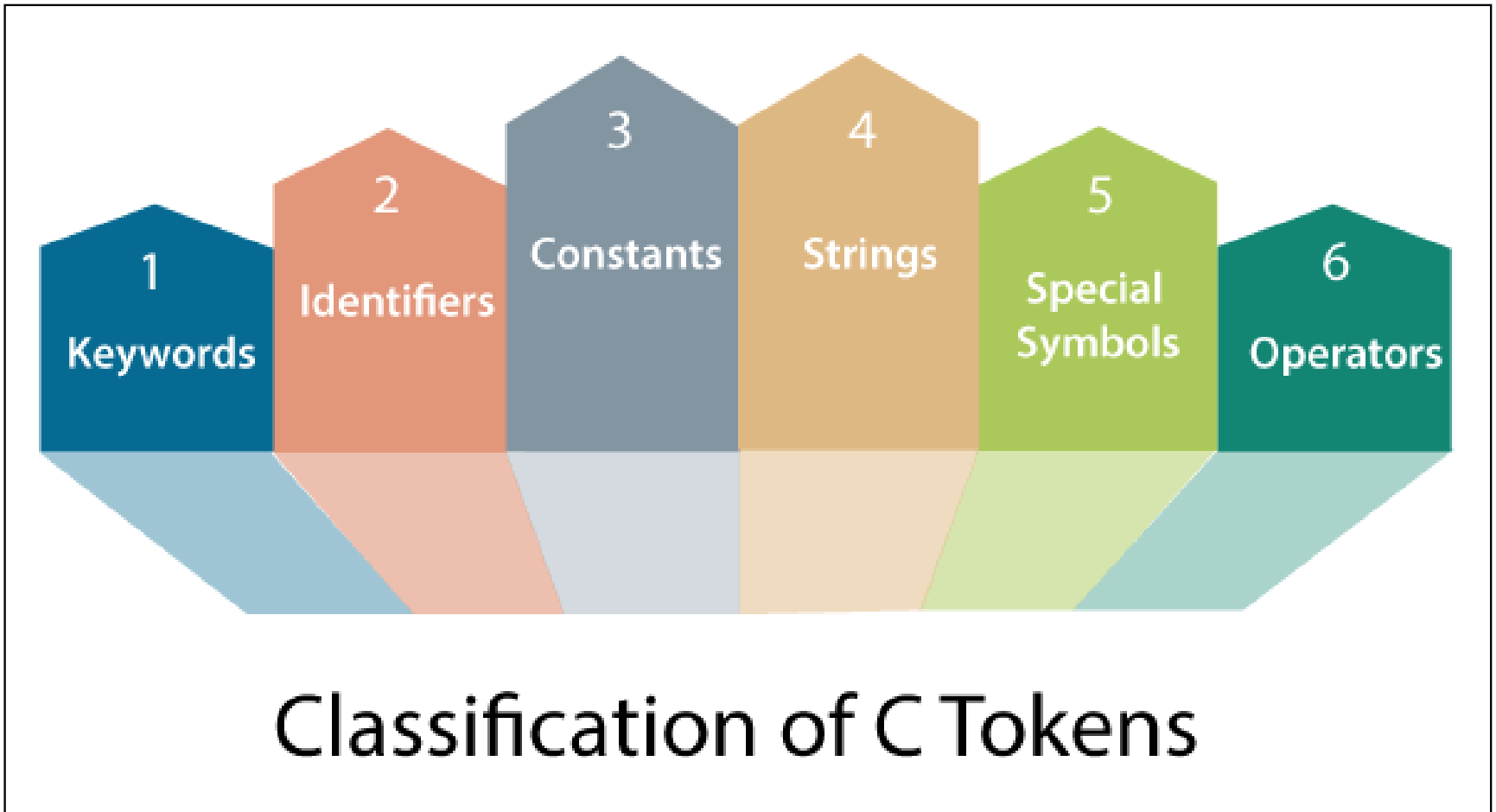
Line 5: return 0 ends the main() function.

Line 6: Do not forget to add the closing curly bracket } to actually end the main function.

C Tokens

Tokens are the smallest elements of a program, which are meaningful to the compiler.

- We can define the token as the smallest individual element in C.
- For `example, we cannot create a sentence without using words; similarly, we cannot create a program in C without using tokens in C.
- Therefore, we can say that tokens in C is the building block or the basic component for creating a program in C language.



Classification of C Tokens

1. Keywords :

Keywords in C can be defined as the **pre-defined** or the **reserved words** having its own importance, and each keyword has its own functionality.

Since keywords are the pre-defined words used by the compiler, so they cannot be used as the variable names.

If the keywords are used as the variable names, it means that we are assigning a different meaning to the keyword, which is not allowed.

C language supports 32 keywords given below:

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

2. Constant :

A constant is **a name given to the variable whose values can't be altered or changed**.

A constant is very similar to variables in the C programming language, but it can hold only a single variable during the execution of a program.

Constant	Example
Integer constant	10, 11, 34, etc.
Floating-point constant	45.6, 67.8, 11.2, etc.
Octal constant	011, 088, 022, etc.
Hexadecimal constant	0x1a, 0x4b, 0x6b, etc.
Character constant	'a', 'b', 'c', etc.
String constant	"java", "c++", ".net", etc.

3. **Strings :**

Strings in C are enclosed within double quotes, while characters are enclosed within single characters.

The size of a string is a number of characters that the string contains.

- char a[] = "javatpoint"; // The compiler allocates the memory at the run time.
- char a[10] = {'j','a','v','a','t','p','o','i','n','t','\0'}; // String is represented in the form of characters.

4. **Identifiers :**

Identifiers in C are used for naming variables, functions, arrays, structures, etc. Identifiers in C are the user-defined words. It can be composed of uppercase letters, lowercase letters, underscore, or digits, but the starting letter should be either an underscore or an alphabet. Identifiers cannot be used as keywords.

- The first character of an identifier should be either an alphabet or an underscore, and then it can be followed by any of the character, digit, or underscore.
- It should not begin with any numerical digit.
- In identifiers, both uppercase and lowercase letters are distinct. Therefore, we can say that identifiers are case sensitive.
- Commas or blank spaces cannot be specified within an identifier.
- Keywords cannot be represented as an identifier.
- The length of the identifiers should not be more than 31 characters.
- Identifiers should be written in such a way that it is meaningful, short, and easy to read.
- Identifiers must be unique

  For example,

  int student; float marks;

Here, student and marks are identifiers

5. Variable :

A **variable** is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times.

- It is a way to represent memory location through symbol so that it can be easily identified.
- syntax to declare a variable: type variable_list;
- The example of declaring the variable is given below:

1. **int** a;
2. **float** b;
3. **char** c;

- Here, a, b, c are variables. The int, float, char are the data types.
- We can also provide values while declaring the variables as given below:

1. **int** a=10,b=20;//declaring 2 variable of integer type
2. **float** f=20.8;
3. **char** c='A';

Rules for defining variables

- A variable can have alphabets, digits, and underscore.
- A variable name can start with the alphabet, and underscore only. It can't start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.
- **Valid variable names:**
  1. **int** a;
  2. **int** _ab;
  3. **int** a30;
- **Invalid variable names:**
  1. **int** 2;
  2. **int** a b;
  3. **int long**;

Operators

- An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.
- There are following types of operators to perform different types of operations in C language.

1. Arithmetic Operators
2. Relational Operators
3. Shift Operators
4. Logical Operators
5. Bitwise Operators
6. Ternary or Conditional Operators
7. Assignment Operator
8. Misc Operator

Arithmetic Operators

Operator	Description	Example
+	Adds two operands.	A + B = 30
−	Subtracts second operand from the first.	A − B = -10
*	Multiplies both operands.	A * B = 200
/	Divides numerator by de-numerator.	B / A = 2
%	Modulus Operator and remainder of after an integer division.	B % A = 0
++	Increment operator increases the integer value by one.	A++ = 11
--	Decrement operator decreases the integer value by one.	A-- = 9

Relational Operators

Operator	Description	Example
==	Checks if the values of two operands are equal or not. If yes, then the condition becomes true.	(A == B) is not true.
!=	Checks if the values of two operands are equal or not. If the values are not equal, then the condition becomes true.	(A != B) is true.
>	Checks if the value of left operand is greater than the value of right operand. If yes, then the condition becomes true.	(A > B) is not true.
<	Checks if the value of left operand is less than the value of right operand. If yes, then the condition becomes true.	(A < B) is true.
>=	Checks if the value of left operand is greater than or equal to the value of right operand. If yes, then the condition becomes true.	(A >= B) is not true.
<=	Checks if the value of left operand is less than or equal to the value of right operand. If yes, then the	(A <= B) is true.

Logical Operators

Operator	Description	Example
&&	Called Logical AND operator. If both the operands are non-zero, then the condition becomes true.	(A && B) is false.
||	Called Logical OR Operator. If any of the two operands is non-zero, then the condition becomes true.	(A || B) is true.
!	Called Logical NOT Operator. It is used to reverse the logical state of its operand. If a condition is true, then Logical NOT operator will make it false.	!(A && B) is true

Bitwise Operators

Operator	Description	Example
&	Binary AND Operator copies a bit to the result if it exists in both operands.	(A & B) = 12, i.e., 0000 1100
|	Binary OR Operator copies a bit if it exists in either operand.	(A | B) = 61, i.e., 0011 1101
^	Binary XOR Operator copies the bit if it is set in one operand but not both.	(A ^ B) = 49, i.e., 0011 0001
~	Binary One's Complement Operator is unary and has the effect of 'flipping' bits.	(~A) = ~(60), i.e,. -0111101
<<	Binary Left Shift Operator. The left operands value is moved left by the number of bits specified by the right operand.	A << 2 = 240 i.e., 1111 0000
>>	Binary Right Shift Operator. The left operands value is moved right by the number of bits specified by the right operand.	A >> 2 = 15 i.e., 0000 1111

Assignment Operators

Operator	Description	Example
=	Simple assignment operator. Assigns values from right side operands to left side operand	C = A + B will assign the value of A + B to C
+=	Add AND assignment operator. It adds the right operand to the left operand and assign the result to the left operand.	C += A is equivalent to C = C + A
-=	Subtract AND assignment operator. It subtracts the right operand from the left operand and assigns the result to the left operand.	C -= A is equivalent to C = C - A
*=	Multiply AND assignment operator. It multiplies the right operand with the left operand and assigns the result to the left operand.	C *= A is equivalent to C = C * A
/=	Divide AND assignment operator. It divides the left operand with the right operand and assigns the result to the left operand.	C /= A is equivalent to C = C / A

Misc Operators ↦ sizeof & ternary

Operator	Description	Example
sizeof()	Returns the size of a variable.	sizeof(a), where a is integer, will return 4.
&	Returns the address of a variable.	&a; returns the actual address of the variable.
*	Pointer to a variable.	*a;
? :	Conditional Expression.	If Condition is true ? then value X : otherwise value Y

%=	Modulus AND assignment operator. It takes modulus using two operands and assigns the result to the left operand.	C %= A is equivalent to C = C % A
<<=	Left shift AND assignment operator.	C <<= 2 is same as C = C << 2
>>=	Right shift AND assignment operator.	C >>= 2 is same as C = C >> 2
&=	Bitwise AND assignment operator.	C &= 2 is same as C = C & 2
^=	Bitwise exclusive OR and assignment operator.	C ^= 2 is same as C = C ^ 2
|=	Bitwise inclusive OR and assignment operator.	C |= 2 is same as C = C | 2

Hierachy of Operators (Precedence of Operators in C)

- The precedence of operator species that which operator will be evaluated first and next. The associativity specifies the operator direction to be evaluated; it may be left to right or right to left.
- Let's understand the precedence by the example given below:

```
int value=10+20*10;
```

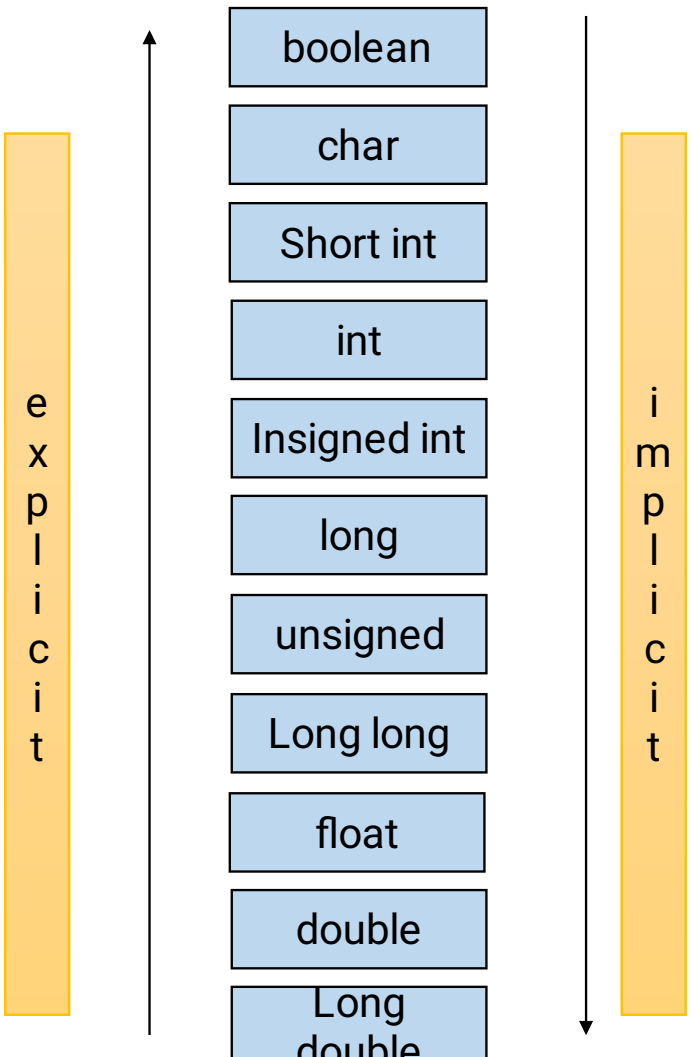
- Certain operators have higher precedence than others; for example, the multiplication operator has a higher precedence than the addition operator.
- For example, x = 7 + 3 * 2; here, x is assigned 13, not 20 because operator * has a higher precedence than +, so it first gets multiplied with 3*2 and then adds into 7.

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

Type Casting in C

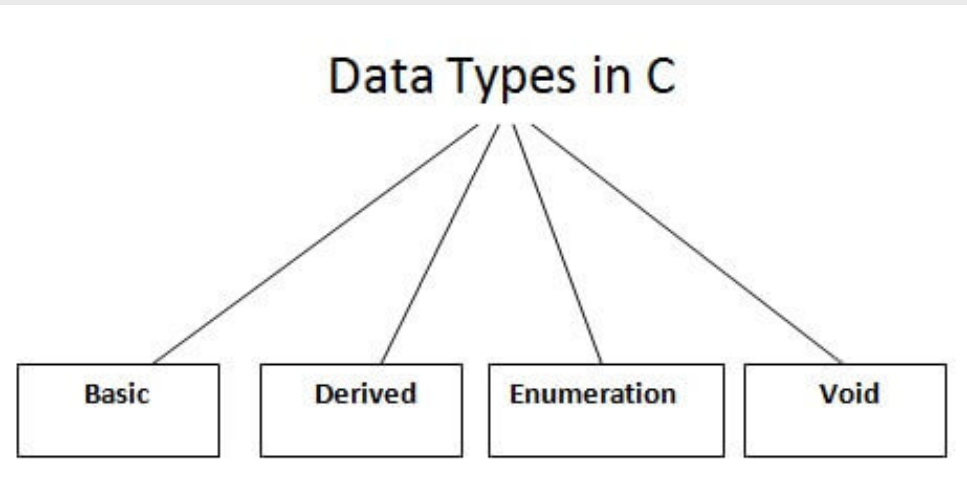
- Typecasting is a way to convert one data type into other.
- In C language, we use cast operator for typecasting which is denoted by (type).
- In type casting, the compiler automatically changes one data type to another one depending on what we want the program to do.
- Note: It is always recommended to convert the lower value to higher for avoiding data loss.
- There are two types of type conversion:

1. **Implicit Type Conversion**
2. **Explicit Type Conversion**


explicit
boolean
char
Short int
int
Insigned int
long
unsigned
Long long
float
double
Long double
implicit

Data Types in C

- A data type specifies the type of data that a variable can store such as integer, floating, character, etc.



Basic Data Types

- The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals.
- The memory size of the basic data types may change according to 32 or 64-bit operating system.
- Let's see the basic data types. Its size is given **according to 32-bit architecture**.

Data Types	Memory Size	Range
char	1 byte	−128 to 127
signed char	1 byte	−128 to 127
unsigned char	1 byte	0 to 255
short	2 byte	−32,768 to 32,767
signed short	2 byte	−32,768 to 32,767
unsigned short	2 byte	0 to 65,535
int	2 byte	−32,768 to 32,767
signed int	2 byte	−32,768 to 32,767
unsigned int	2 byte	0 to 65,535
short int	2 byte	−32,768 to 32,767
signed short int	2 byte	−32,768 to 32,767
unsigned short int	2 byte	0 to 65,535

long int	4 byte	-2,147,483,648 to 2, 147,483,647
signed long int	4 byte	-2,147,483,648 to 2, 147,483,647
unsigned long int	4 byte	0 to 4,294,967,295
float	4 byte	-
long	8 byte	-
long double	10 byte	-

C Preprocessor

- All preprocessor commands begin with a hash symbol (#).
- It must be the first nonblank character, and for readability, a preprocessor directive should begin in the first column.
- The C preprocessor is a micro processor that is used by compiler to transform your code before compilation.
- It is called micro preprocessor because it allows us to add macros.
- Let's see a list of preprocessor directives.
- #include
- #define
- #undef
- #ifdef
- #ifndef
- #if
- #else
- #elif
- #endif
- #error
- #pragma

#define

- The #define preprocessor directive is used to define constant or micro substitution. It can use any basic data type.
- #define PI 3.14
- Let's see an example of #define to define a constant.

```
#include <stdio.h>
#define PI 3.14
main() {
   printf("%f",PI);
}
```

C #undef

- The #undef preprocessor directive is used to undefine the constant or macro defined by #define.
- Let's see a simple example to define and undefine a constant.

```
#include <stdio.h>
#define PI 3.14
#undef PI
main()
{
   printf("%f",PI);
}
```

Introduction of logic

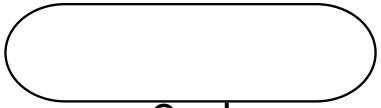
- **a proper or reasonable way of thinking about something.**
- **Here are some tips to improve the logic in your programs and effectively write better code.**

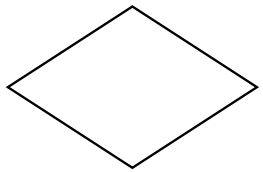
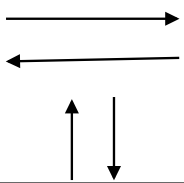
1. Practice writing a lot of code. ...
2. Check solutions by other people. ...
3. Use a pen and paper to work out solutions. ...
4. Keep learning new things. ...
5. Be consistent. ...
6. Face problems head-on. ...
7. Don't lose motivation.

Basics of Flow chart

- A flowchart can be helpful for both writing programs and explaining the program to others.
- Algorithm is a step – by – step procedure which is helpful in solving a problem.
- It makes use of symbols which are connected among them to indicate the flow of information and processing.
- The process of drawing a flowchart for an algorithm is known as “flowcharting”.

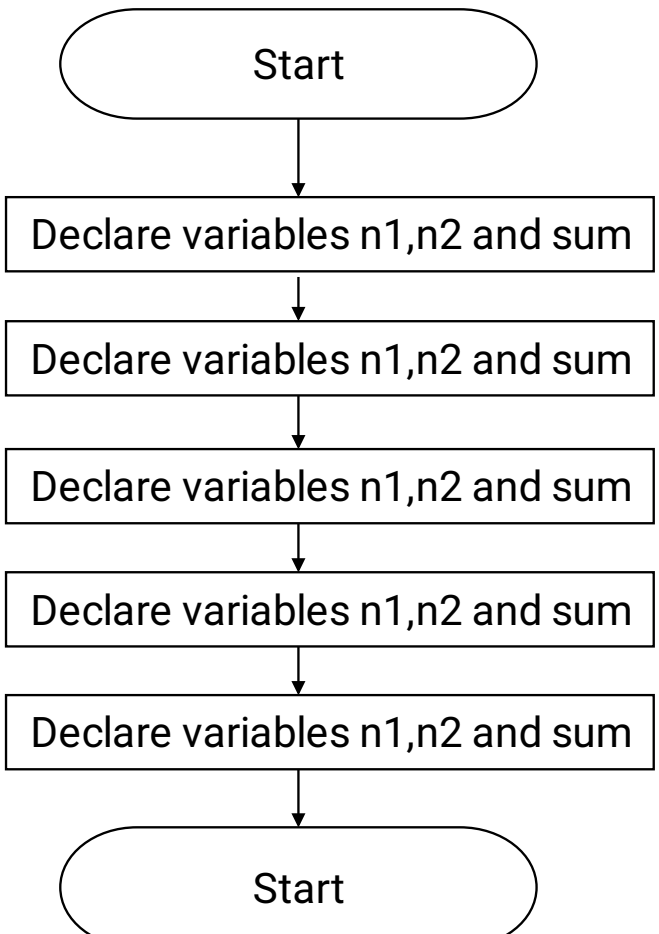
Symbols

Name	Symbol	Purpose
Terminal	 Oval	The oval symbol indicates Start, Stop in a program's logic flow. A pause/halt is generally used in a program logic under some error conditions. Terminal is the first and last symbols in the flowchart.
Input/output	 Parallelogram	Input/output of data. A parallelogram denotes any function of input/ output type. Program instructions that take input from input devices and display output on output devices are indicated with parallelogram in a flowchart.
Process	 Rectangle	Any processing to be performaed can be represented. A box represents arithmetic instructions. All arithmetic processes such as adding, subtracting, multiplication and division are indicated by action

Name	Symbol	Purpose
Decision box	 Diamond	Diamond symbol represents a decision point. Decision based operations such as yes/no question or true/false are indicated by diamond in flowchart.
Connector	 Connector	Used to connect different parts of flowchart
Flow Lines		Join 2 symbols and also represents flow of execution

Example of flowcharts in programming

- 1. Add two numbers entered by the user.



Dry run and its use

- Dry run is a just a technical term. It means that **you have to iterate though each line of your program code**, simply before running the code on the machine we should run it on the paper.
- You act as the computer – following the instructions of the program, recording the valves of the variables at each stage.
- You can do this with a table.
- The table with have column headed with the names of the variables in the program Each row in the table will be labelled with a line number form the program.
- The entries of each row in the table will be values of the variables after the execution of the statement on that line You don’t need a row for every line in the program, just those lines where some significant change to the state of the program occurs